CEN571/CSE494/CSE598: RC Lab 3
FPGA Architecture and CAD Exploration using VTR

Yaotian Liu (ASU ID: 1229652231), Xiangyu Zhou (ASU ID: 1229983081)

Part 1: Logic Block Architecture Evaluation

Section 4.2:

What technology node (nm) for this architecture?
40 nm technology

Name of the CLB tile
Name: clb

```
<tile name="clb" area="53894">
  <sub_tile name="clb">
    <equivalent_sites>
      <site pb_type="clb" pin_mapping="direct"/>
    </equivalent_sites>
    <input name="I1" num_pins="13" equivalent="full"/>
    <input name="I2" num_pins="13" equivalent="full"/>
    <input name="I3" num_pins="13" equivalent="full"/>
    <input name="I4" num_pins="13" equivalent="full"/>
    <input name="cin" num_pins="1"/>
    <output name="O" num_pins="20" equivalent="none"/>
    <output name="cout" num_pins="1"/>
    <clock name="clk" num_pins="1"/>
    <fc in_type="frac" in_val="0.15" out_type="frac" out_val="0.10">
      <fc_override port_name="cin" fc_type="frac" fc_val="0"/>
      <fc_override port_name="cout" fc_type="frac" fc_val="0"/>
    </fc>
    <pinlocations pattern="spread"/>
  </sub_tile>
</tile>
```

Name and width of input and output ports of the CLB
Input port names (+width):

- I1, I2, I3, I4, each with 13 pins
- cin with 1 pin

Output port names (+width):

- O with 20 pins.
- cout with 1 pin.

Name and number of BLEs in this CLB
Name: fle
Number: 10

Names of the modes supported by the CLB

- n2_lut5

    - blut5
    - arithmetic
- n1_lut6

What is the setup time and the clock-to-q delay of the FF in the BLE

- T_setup value="66e-12"
- T_clock_to_Q max="124e-12"

Does this CLB have a local interconnect? If so, what is the type of local interconnect used in this CLB?
Answer: 50% depopulated crossbar

Name of the memory tile
Name: memory

What are the various modes supported by this memory
Single port mode (512 × 64) – 9 address pins and 64 data pins
Single port mode (1024 × 32) – 10 address pins and 32 data pins
Single port mode (2048 × 16) – 11 address pins and 16 data pins
Single port mode (4096 × 8) – 12 address pins and 8 data pins
Single port mode (8192 × 4) – 13 address pins and 4 data pins
Single port mode (16384 × 2) – 14 address pins and 2 data pins
Single port mode (32768 × 1) – 15 address pins and 1 data pin

Dual port mode (1024 × 32) –
addr1 and addr2 have 10 pins each, two data lines have 32 pins each
Dual port mode (2048 × 16) –
addr1 and addr2 have 11 pins each, two data lines have 16 pins each
Dual port mode (4096 × 8) –
addr1 and addr2 have 12 pins each, two data lines have 8 pins each
Dual port mode (8192 × 4) –
addr1 and addr2 have 13 pins each, two data lines have 4 pins each
Dual port mode (16384 × 2) –
addr1 and addr2 have 14 pins each, two data lines have 2 pins each
Dual port mode (32768 × 1) –
addr1 and addr2 have 15 pins each, two data lines have 1 pin each

Name of the DSP tile (is it really called DSP?)
Name: mult_36

What are the various modes supported by this DSP?

- two_divisible_mult_18x18
    - two_mult_9x9
    - mult_18x18

- mult_36x36

What is the grid size (number of tiles vertically and horizontally) of the FPGA? Is this specified in the architecture file?
Answer: The grid size is not explicitly fixed in the architecture file.

What is the routing channel width in the FPGA? Is this specified in the architecture file?
Answer: not explicitly specified

Wire segments of which length present in the routing channel?
Answer: 4

What of the value of Fs, Fcin and Fcout in this architecture?
Fs = 3, Fcin = 0.15, Fcout = 0.10

What does the word “depop” mean? You may have to search in the file to find the answer.
Answer: depop50 indicates a crossbar with about 50% population, reducing local-routing switch count relative to a full crossbar while preserving logical equivalence through the remaining connections.

Extra Credit (optional)
Paste the figure here

Section 4.5

What is the name of the synthesis tool, and the file name of the log file generated by the synthesis tool? What is the command line that was used to run this tool by going through this logfile? What is the name of the output file from this stage of the flow?

Synthesis Tool: PARMYS

Log file: parmys.out

Command line used to run this tool: /packages/apps/vtr/8.0.0-git/build/bin/yosys -c synthesis.tcl

Name of output file: <CIRCUIT>.parmys.blif

What is the name of the logic optimization and technology mapping tool, and the file name of the log file generated by the tool? What is the command line that was used to run this tool by going through this logfile? What is the name of the output file from this stage of the flow?

Logic optimization and technology mapping tool: ABC

File name of log file generated by tool: abc0.out

Command line used to run this tool by going through logfile: `/usr/bin/env time -v /packages/apps/vtr/8.0.0-git/abc/abc -c echo ""; echo "Load Netlist"; echo "============"; read 0_diffeq2.parmys.blif; time; echo ""; echo "Circuit Info"; echo "==========="; print_stats; print_latch; time; echo ""; echo "LUT Costs"; echo "========="; print_lut; time; echo ""; echo "Logic Opt + Techmap"; echo "==================="; strash; ifraig -v; scorr -v; dc2 -v; dch -f; if -K 6 -v; mfs2 -v; print_stats; time; echo ""; echo "Output Netlist"; echo "==============="; write_hie 0_diffeq2.parmys.blif 0_diffeq2.raw.abc.blif; time;`

Name of output file from this stage of the flow: <CIRCUIT>.abc.blif

What is the name of the tool that does packing, placement and routing? What is the file name of the log file generated by the tool? What is the command line that was used to run this tool by going through this logfile? What is the name of the output file from this stage of the flow

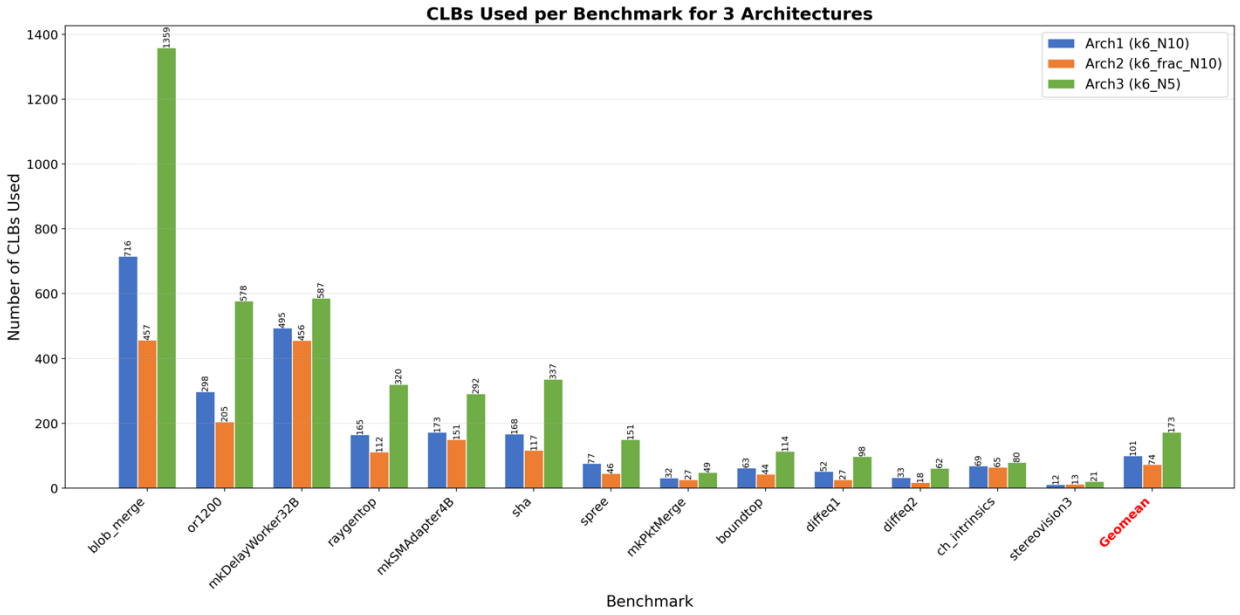
Packing, Placement, Routing: VPR

File name of log file: vpr.out

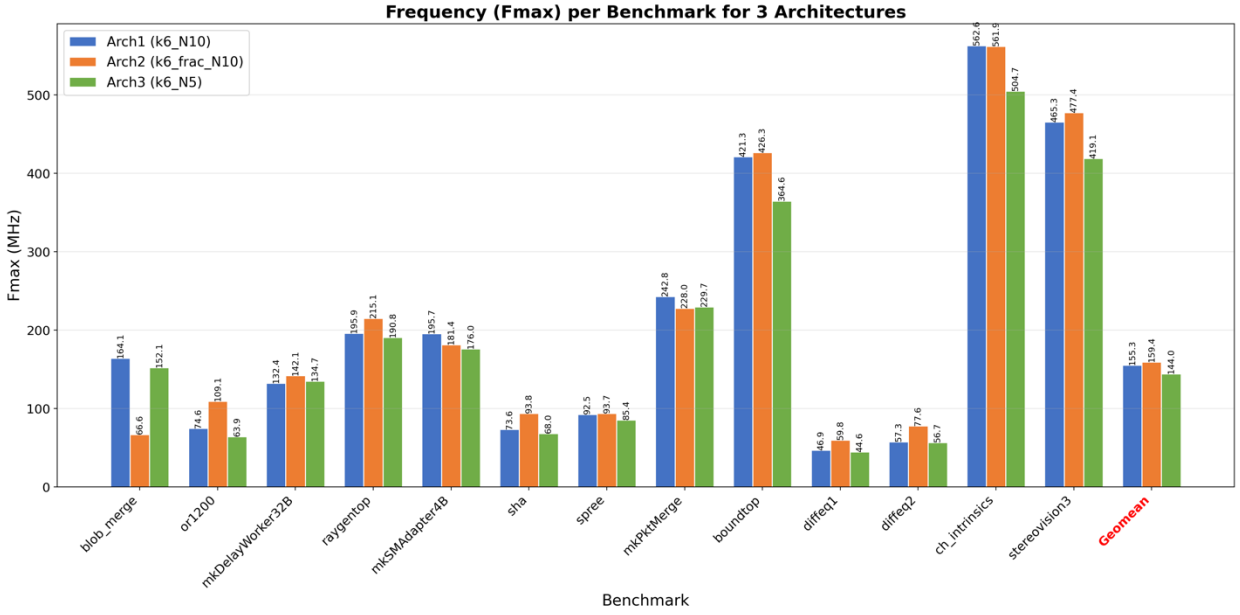
Command line to run this tool: `/packages/apps/vtr/8.0.0-git/vpr/vpr k6_N10_mem32K_40nm.xml <CIRCUIT> --circuit_file <CIRCUIT>.pre-vpr.blif --route_chan_width 300`

Name of the output file from this stage of flow: <CIRCUIT>.net (packing), <CIRCUIT>.place (placement), <CIRCUIT>.route (routing)

1. Draw a chart comparing the area (CLBs used) by each benchmark for all 3 architectures. Along with each benchmark, include a data point for the geometric mean of the CLB count across all benchmarks.



2. Draw a chart comparing the frequency obtained by each benchmark for all 3 architectures. Along with each benchmark, include a data point for the geometric mean of the frequency across all benchmarks.



3. Explain the reason behind what you observe.

As for CLBs:

Arch2 uses the fewest CLBs because its fracturable LUTs allow a single 6-LUT to be split into two independent 5-LUTs. Additionally, carry chains provide dedicated hardware for arithmetic, absorbing adder/counter logic that would otherwise consume extra LUTs. The result is significantly better logic utilization per CLB.

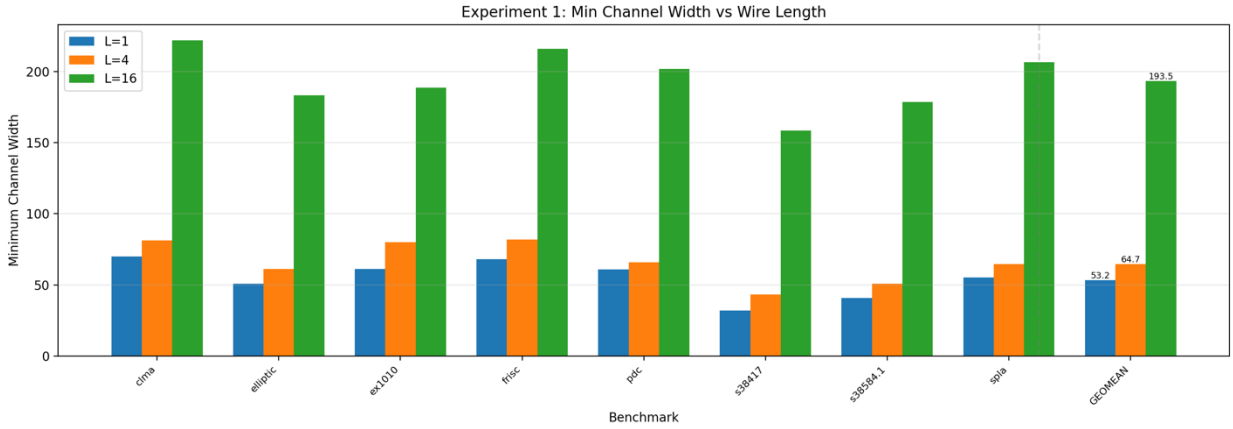
Arch3 uses the most CLBs because each CLB contains only 5 BLEs (vs. 10 in Arch1/Arch2). Since each CLB holds half the logic capacity, roughly 2x more CLBs are needed.

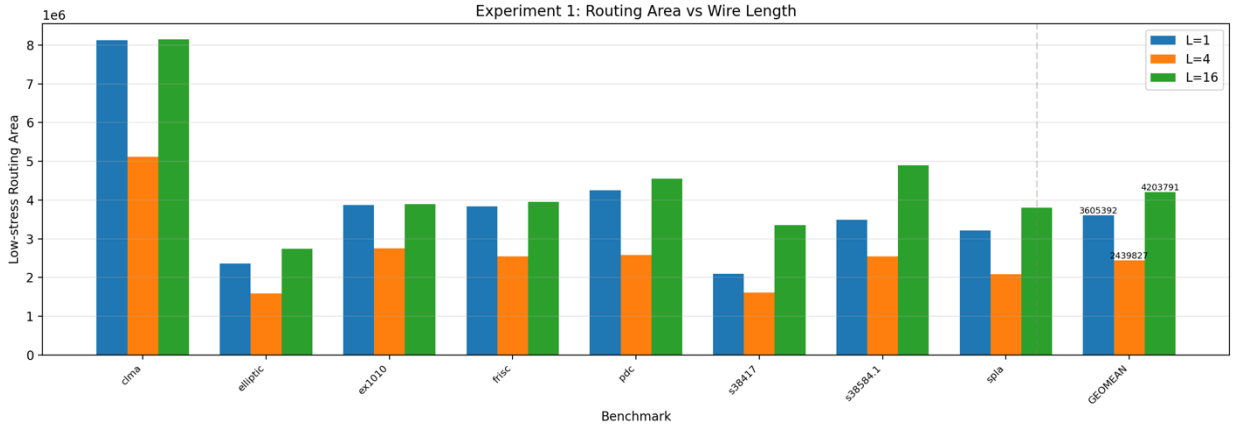
For frequency:
Arch2 achieves the highest frequency primarily due to its carry chains, which provide fast dedicated paths for arithmetic operations that bypass the slower general-purpose routing fabric. Arch3 has the lowest frequency because smaller CLBs (N=5) mean more signals must travel between CLBs through the general-purpose routing fabric.

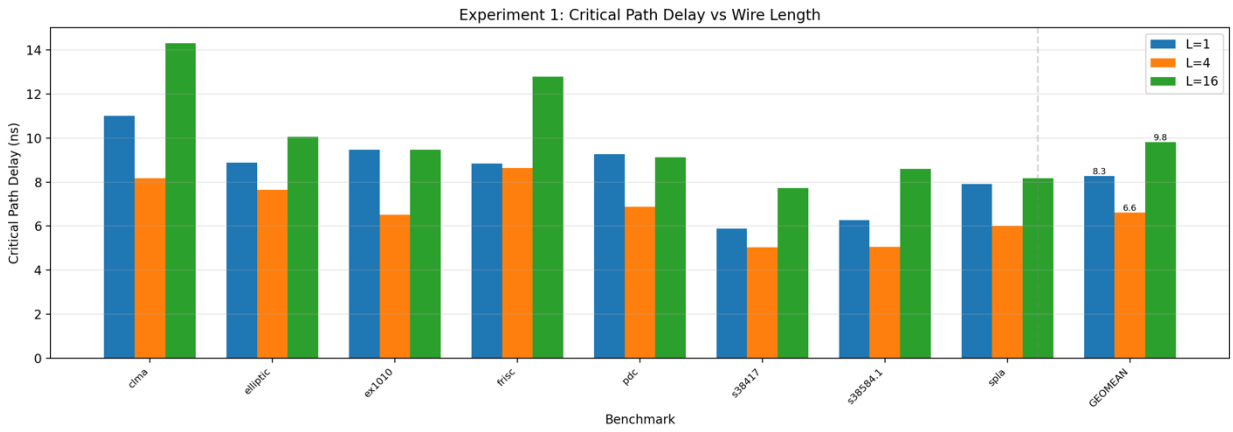
Part 2: Routing Architecture Evaluation

Section 5.3

1. Draw a chart or table showing the geometric average (over the 8 benchmarks). This will show how the 3 metrics listed above change when you change the wire length.







	Minimum Channel Width (Geo mean)	Low-stress routing area (Geo mean)	Low-stress routing critical path delay (Geo mean)
Wire length 1	53.2	3,605,392	8.29 ns
Wire length 4	64.7	2,439,827	6.62 ns
Wire length 16	193.5	4,203,791	9.82 ns

The above data was taken from the below tables:

For wire length == 1
Each metric is averaged (arithmetic mean) across 3 seeds (seed=1, seed=2, seed=3)

	Minimum Channel Width (3 seeds)	Low-stress routing area (3 seeds)	Low-stress routing critical path delay (3 seeds)
clma.blif	70.0	8,131,720	11.018 ns
elliptic.blif	50.7	2,365,670	8.883 ns
ex1010.blif	61.3	3,870,870	9.477 ns
frisc.blif	68.0	3,837,770	8.841 ns
pdc.blif	60.7	4,251,623	9.276 ns
s38417.blif	32.0	2,095,320	5.883 ns
s38584.1.blif	40.7	3,493,160	6.271 ns
spla.blif	55.3	3,210,500	7.917 ns
GEOMETRIC MEAN	53.2	3,605,392	8.286 ns

For wire length == 4
Each metric is averaged (arithmetic mean) across 3 seeds (seed=1, seed=2, seed=3)

	Minimum Channel Width (3 seeds)	Low-stress routing area (3 seeds)	Low-stress routing critical path delay (3 seeds)
clma.blif	81.3	5,119,740	8.176 ns
elliptic.blif	61.3	1,584,343	7.657 ns
ex1010.blif	80.0	2,757,227	6.513 ns
frisc.blif	82.0	2,543,320	8.649 ns

pdc.blif	66.0	2,581,880	6.885 ns
s38417.blif	43.3	1,614,713	5.031 ns
s38584.1.blif	50.7	2,540,447	5.061 ns
spla.blif	64.7	2,084,297	5.999 ns
GEOMETRIC MEAN	64.7	2,439,827	6.624 ns

For wire length == 16
Each metric is averaged (arithmetic mean) across 3 seeds (seed=1, seed=2, seed=3)

	Minimum Channel Width (3 seeds)	Low-stress routing area (3 seeds)	Low-stress routing critical path delay (3 seeds)
clma.blif	222.0	8,149,800	14.310 ns
elliptic.blif	183.3	2,740,070	10.050 ns
ex1010.blif	188.7	3,896,603	9.470 ns
frisc.blif	216.0	3,953,150	12.789 ns
pdc.blif	202.0	4,547,695	9.127 ns
s38417.blif	158.7	3,348,213	7.734 ns
s38584.1.blif	178.7	4,893,513	8.609 ns
spla.blif	206.7	3,805,097	8.168 ns
GEOMETRIC MEAN	193.5	4,203,791	9.820 ns

2. Explain your results. Why do the metrics vary when the wirelength is changed in the way they do?
Wire length L=1 requires the lowest minimum channel width (53.2) because short wires offer the finest granularity of routing. However, L=1 has the highest routing area (3.61M) and second-highest delay (8.29 ns) because signals must traverse many programmable switch points to cover any significant distance, each adding resistance, capacitance, and switch area.

Wire length L=4 achieves the best balance: moderate minimum channel width (64.7), the lowest routing area (2.44M), and the lowest critical path delay (6.62 ns). L=4 wires can span multiple CLB tiles in a single hop, reducing the number of switch points on a typical path.

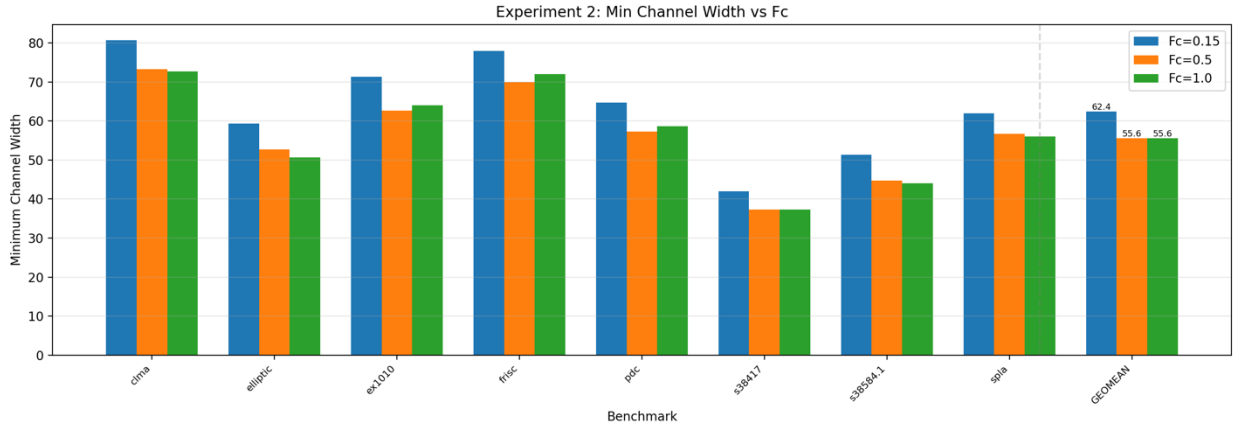
Wire length L=16 has by far the highest minimum channel width (193.5) because each wire spans 16 tiles—most connections are local, so these long wires are underutilized, wasting tracks. Routing area (4.20M) and delay (9.82 ns) are also the worst.

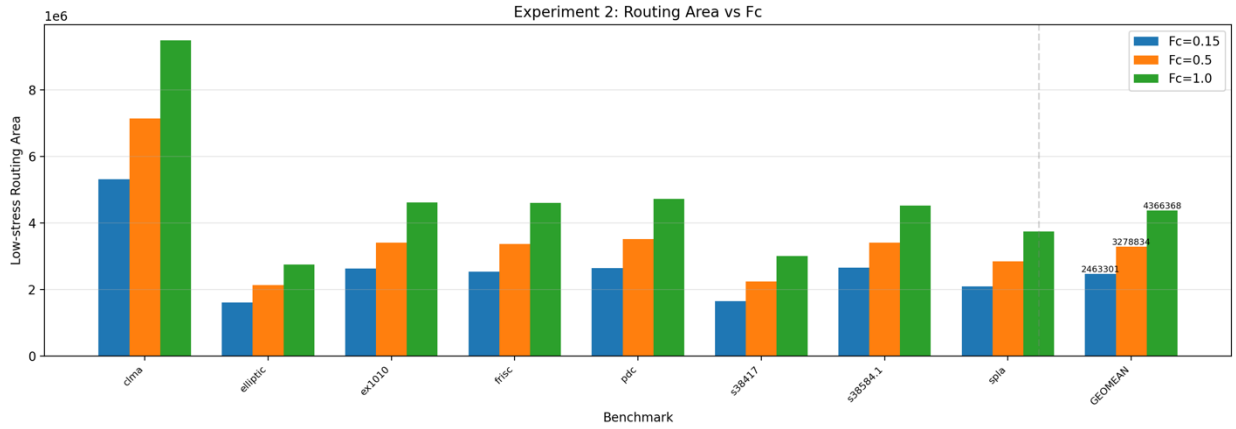
3. Which routing wire length do you consider the best (gives the best combination of area and delay)?

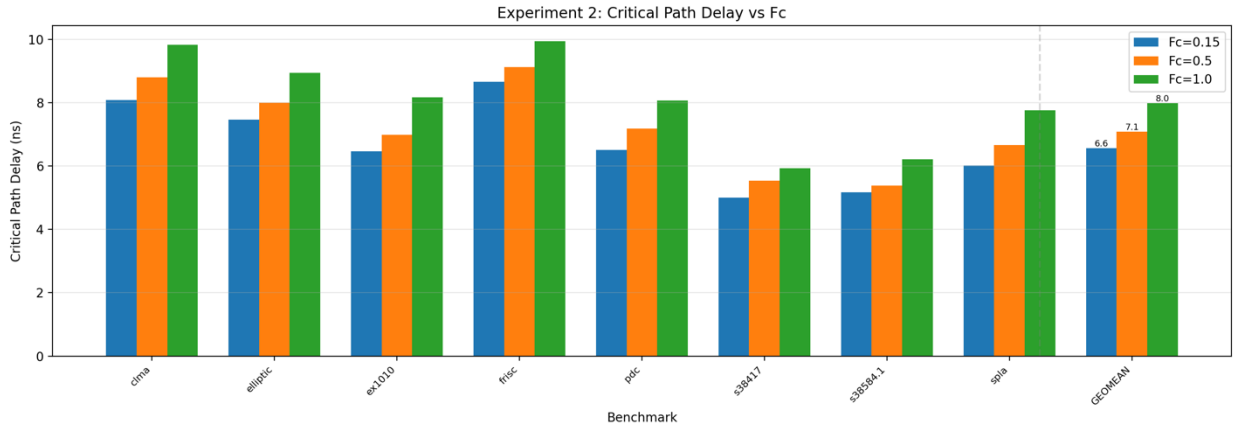
Wire length L=4 gives the best combination of area and delay. It achieves the lowest routing area (2.44M, 32% less than L=1) and the lowest critical path delay (6.62 ns, 20% less than L=1). This matches the well-known result that L=4 is near-optimal for island-style FPGAs, as it balances the locality of most connections with the need to occasionally reach farther-away blocks.

Section 5.4

1. Draw a chart showing the geometric average (over the 8 benchmarks) of the 3 metrics for the various values of Fcin and Fcout.







	Min Channel Width (Geo mean)	Low-stress routing area (Geo mean)	Low-stress critical path delay (Geo mean)
Fcin = Fcout = 0.15	62.4	2,463,301	6.56 ns
Fcin = Fcout = 0.5	55.6	3,278,834	7.09 ns
Fcin = Fcout = 1.0	55.6	4,366,368	7.98 ns

The above data was taken from the below tables:

For Fcin = Fcout = 0.15
Each metric is averaged (arithmetic mean) across 3 seeds (seed=1, seed=2, seed=3)

	Minimum Channel Width (3 seeds)	Low-stress routing area (3 seeds)	Low-stress routing critical path delay (3 seeds)
clma.blif	80.7	5,312,190	8.081 ns
elliptic.blif	59.3	1,602,670	7.467 ns
ex1010.blif	71.3	2,623,547	6.460 ns
frisc.blif	78.0	2,529,773	8.652 ns
pdc.blif	64.7	2,639,920	6.509 ns
s38417.blif	42.0	1,641,797	4.996 ns
s38584.1.blif	51.3	2,648,750	5.178 ns
spla.blif	62.0	2,089,780	6.020 ns
GEOMETRIC MEAN	62.4	2,463,301	6.557 ns

For Fcin = Fcout = 0.5
Each metric is averaged (arithmetic mean) across 3 seeds (seed=1, seed=2, seed=3)

	Minimum Channel Width (3 seeds)	Low-stress routing area (3 seeds)	Low-stress routing critical path delay (3 seeds)
clma.blif	73.3	7,133,660	8.793 ns
elliptic.blif	52.7	2,137,007	7.994 ns
ex1010.blif	62.7	3,406,873	6.986 ns
frisc.blif	70.0	3,371,150	9.124 ns
pdc.blif	57.3	3,513,687	7.178 ns
s38417.blif	37.3	2,243,917	5.534 ns
s38584.1.blif	44.7	3,405,873	5.385 ns
spla.blif	56.7	2,841,217	6.669 ns
GEOMETRIC MEAN	55.6	3,278,834	7.091 ns

For Fcin = Fcout = 1.0
Each metric is averaged (arithmetic mean) across 3 seeds (seed=1, seed=2, seed=3)

	Minimum Channel Width (3 seeds)	Low-stress routing area (3 seeds)	Low-stress routing critical path delay (3 seeds)
clma.blif	72.7	9,487,180	9.825 ns
elliptic.blif	50.7	2,746,973	8.945 ns
ex1010.blif	64.0	4,609,080	8.173 ns
frisc.blif	72.0	4,598,650	9.939 ns
pdc.blif	58.7	4,721,393	8.069 ns
s38417.blif	37.3	2,998,120	5.928 ns
s38584.1.blif	44.0	4,517,290	6.214 ns
spla.blif	56.0	3,740,500	7.756 ns
GEOMETRIC MEAN	55.6	4,366,368	7.981 ns

2. Explain your results – why do Wmin, routing area and critical path delay vary (or not vary) with Fcin and Fcout in the way they do.
Minimum channel width decreases as Fc increases (62.4 → 55.6 → 55.6). Higher Fc means each CLB pin connects to a larger fraction of the routing tracks, giving the router more flexibility.

The improvement saturates from Fc=0.5 to Fc=1.0 (both 55.6) because once enough tracks are reachable per pin, additional connections provide diminishing returns.

Routing area increases significantly with Fc (2.46M → 3.28M → 4.37M, a 77% increase from Fc=0.15 to Fc=1.0). Even though higher Fc reduces the minimum channel width slightly, the connection box itself becomes much larger.

Critical path delay also increases with Fc (6.56 → 7.09 → 7.98 ns). Larger connection boxes add parasitic capacitance to the routing tracks.

The key insight is that Fc=0.15 achieves the best area and delay despite requiring slightly more tracks, because the savings from small connection boxes far outweigh the cost of a few extra routing channels.

Extra Credit (optional)

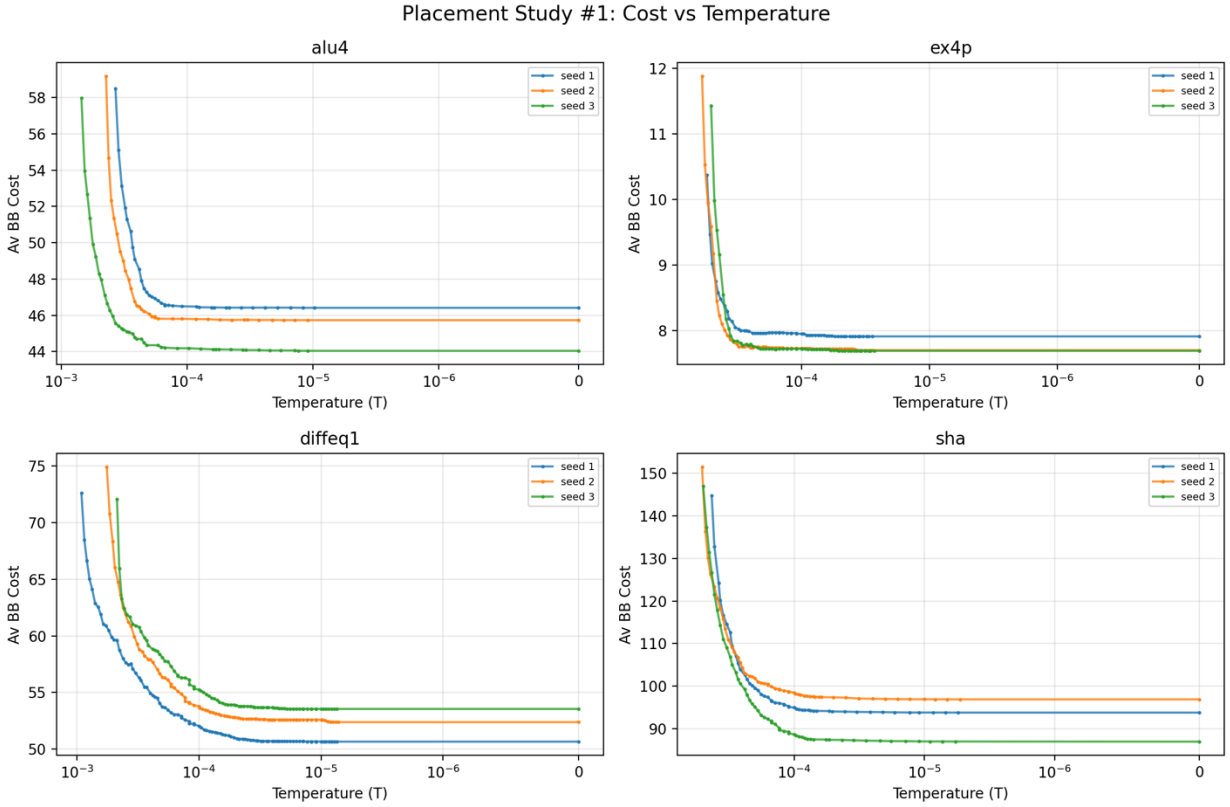
Provide chart/table to compare partial vs. full crossbar

Does the choice between a full crossbar and a partial crossbar impact the channel width and the best choice of Fc?

Part 3: CAD Algorithm Evaluation

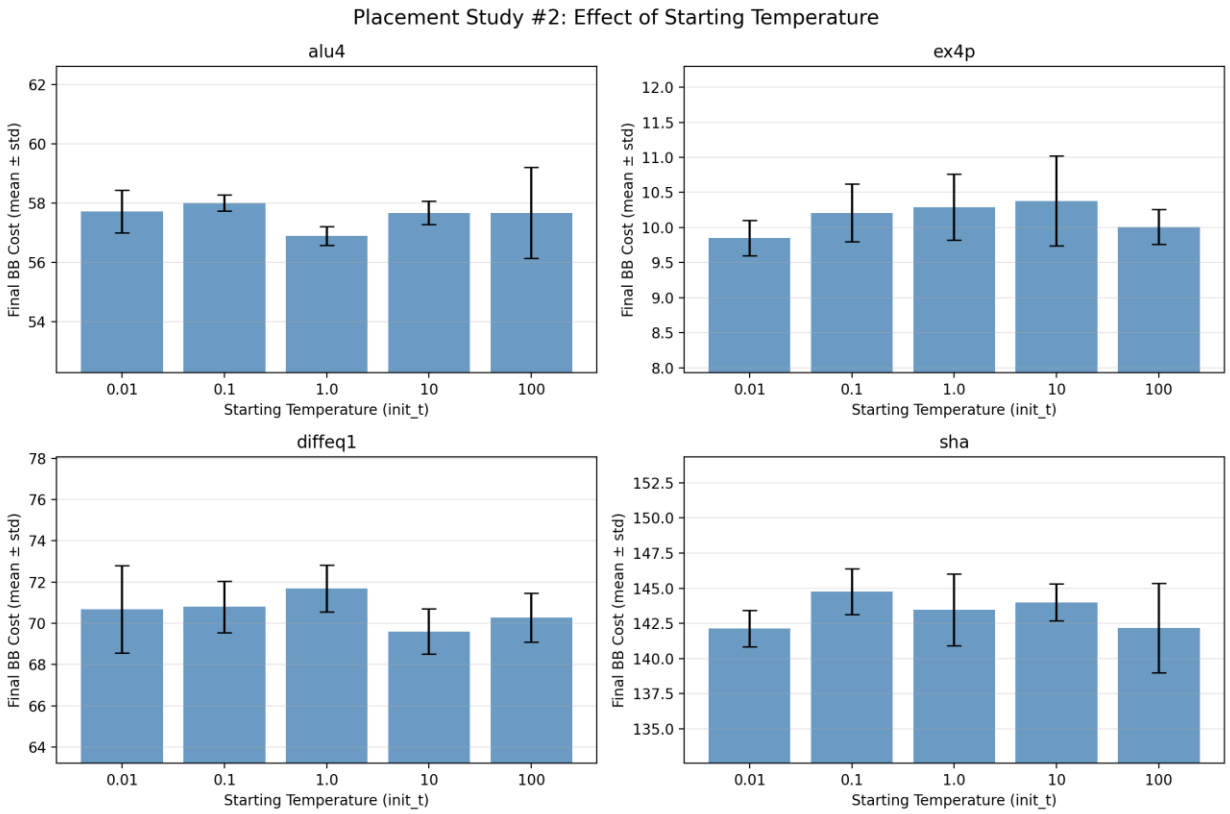
Section 6.3 #1

Plot the score (cost function, “Av BB Cost”) versus the temperature (“T”)



Section 6.3 #2

For each circuit, report the mean and standard deviation of the final score for each starting temperature. You may use a chart or a table.



The starting temperature has limited impact on final placement quality. All init_t values converge to similar final costs, demonstrating the robustness of the simulated annealing algorithm. Lower starting temperatures (0.01, 0.1) tend to produce slightly lower mean costs for most benchmarks. Very high starting temperatures (100.0) increase variance because the algorithm spends many iterations in the high-temperature random-walk phase before reaching the productive cooling regime.

Benchmark	init_t = 0.01	init_t = 0.1	init_t = 1.0	init_t = 10.0	init_t = 100.0
alu4	57.71 ± 0.72	58.00 ± 0.27	56.89 ± 0.32	57.67 ± 0.40	57.67 ± 1.54
ex4p	9.85 ± 0.25	10.21 ± 0.41	10.29 ± 0.47	10.38 ± 0.64	10.01 ± 0.25
diffeq1	70.68 ± 2.13	70.80 ± 1.25	71.69 ± 1.13	69.60 ± 1.10	70.28 ± 1.19
sha	142.14 ± 1.30	144.76 ± 1.63	143.47 ± 2.56	143.99 ± 1.31	142.17 ± 3.20

Section 6.3 #3

Mention the difference in the VPR placement code between the original (this is the VPR code from the VTR installation that you used in earlier parts of the lab) and the modified (this is the VPR code from the VTR installation that was modified for this part (see highlighted text above)).

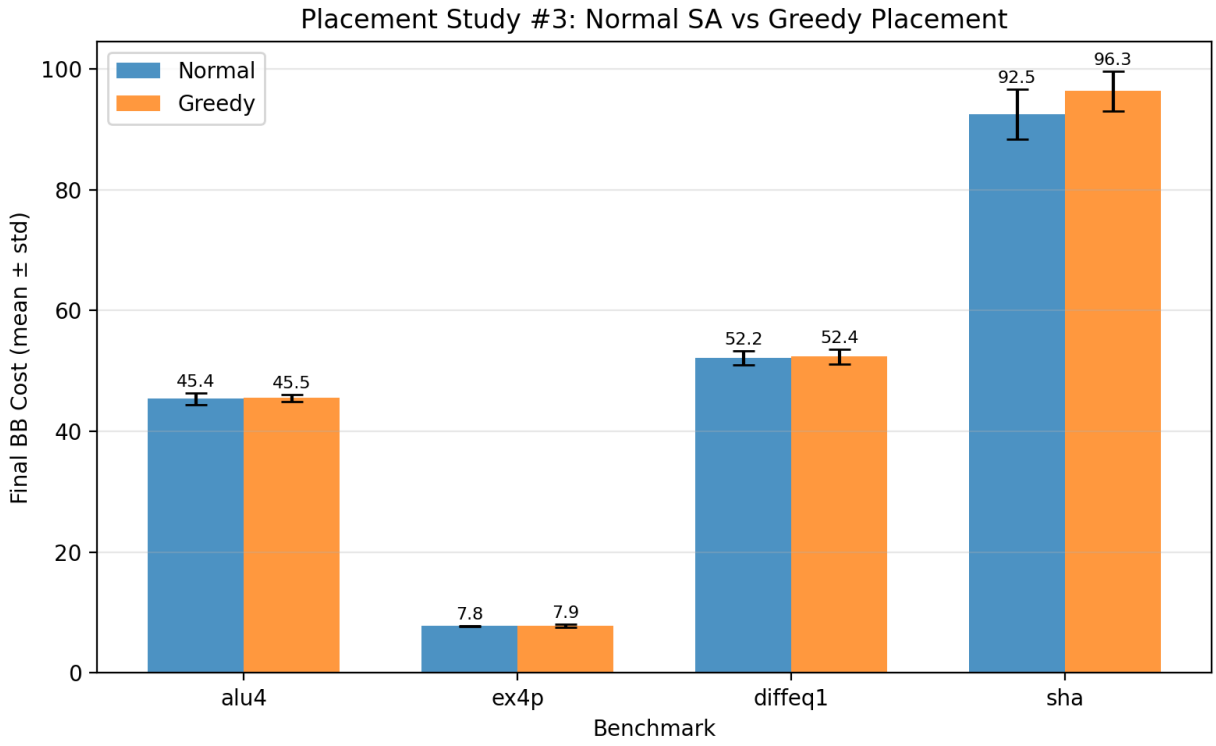
In vpr/src/place/place.cpp, the modified version comments out the Metropolis acceptance criterion:

```
// ORIGINAL (lines 2421-2431):
    if (t == 0.) { return REJECTED; }
    float fnum = vtr::frand();
    float prob_fac = std::exp(-delta_c / t);
    if (prob_fac > fnum) { return ACCEPTED; }

// MODIFIED: All of the above is commented out.
```

The modification removes: (1) the rejection of all moves when T=0, and (2) the probabilistic acceptance of uphill moves. Without this, the placer becomes a greedy/hill-descent algorithm that only accepts moves that reduce cost.

Report the mean and standard deviation of the resulting scores, for each benchmark. Discuss quality differences observed versus the 'normal results'. You may use a chart or a table.



The greedy placer produces slightly worse results than the normal simulated annealing placer for all benchmarks. The difference is small for alu4, ex4p, and diffeq1 (< 1%), but more noticeable for sha (4.1% worse). This is expected: the Metropolis criterion allows simulated annealing to escape local minima by occasionally accepting uphill moves. The greedy approach gets trapped in local minima more easily.

Benchmark	Normal (mean ± std)	Greedy (mean ± std)
alu4	45.39 ± 1.00	45.51 ± 0.56
ex4p	7.77 ± 0.10	7.86 ± 0.27
diffeq1	52.20 ± 1.19	52.40 ± 1.25
sha	92.51 ± 4.15	96.33 ± 3.27

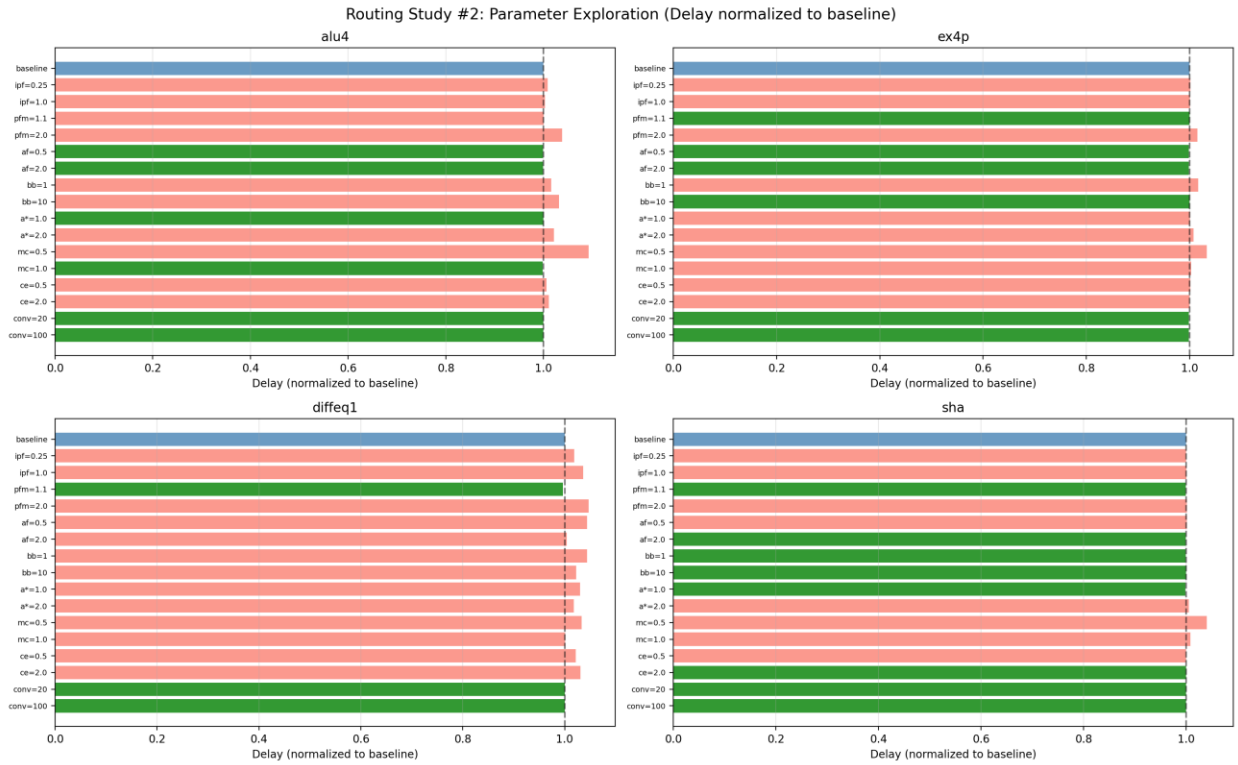
Section 6.4 #1

Report the post-routing critical path delay and the router run-time for each benchmark averaged (arithmetic mean) over the seeds. You may use a chart or a table.

Benchmark	Fixed W	Avg Critical Path Delay (ns)	Avg Wirelength	Avg Router Run-time (s)	Avg Iterations
alu4	40	5.497	6,286	0.14	14.0
ex4p	50	3.064	1,231	0.03	10.7
diffeq1	60	23.424	9,770	0.31	13.7
sha	60	15.425	14,451	0.21	13.0

Section 6.4 #2

Create a chart or table showing the data. Show the data averaged (arithmetic mean) over seeds for each benchmark. Also, show the data averaged (geometric mean) over all benchmarks.



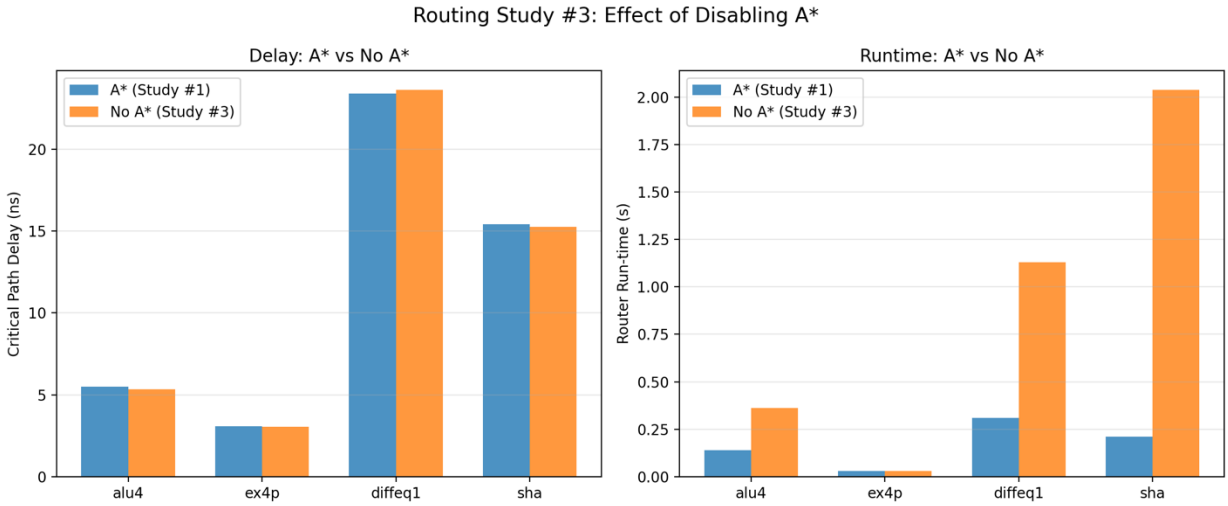
Key observations: max_criticality has the largest effect on delay (5-10% increase at 0.5 vs baseline). pres_fac_mult=1.1 gives slightly better delay but 2-3x more runtime. Other parameters have relatively minor effects. (*max_crit=1.0: diffeq1 has only 1 seed, sha has 2 seeds due to routing failures. Delay units: ns, runtime units: seconds.)

Config	alu4 delay	ex4p delay	diffeq1 delay	sha delay	alu4 rt	ex4p rt	diffeq1 rt	sha rt	GEOMEAN delay (ns)	GEOMEAN runtime (s)
baseline	5.034	2.967	22.669	14.830	0.13	0.03	0.37	0.28	8.418	0.142
init_pres_fac=0.25	5.077	2.971	23.100	14.831	0.15	0.03	0.46	0.38	8.479	0.167
init_pres_fac=1.0	5.050	2.969	23.484	14.838	0.12	0.02	0.35	0.27	8.502	0.123
pres_fac_mult=1.1	5.037	2.967	22.595	14.824	0.21	0.04	0.63	0.49	8.411	0.226
pres_fac_mult=2.0	5.225	3.012	23.728	14.850	0.10	0.02	0.32	0.27	8.629	0.115
acc_fac=0.5	5.030	2.962	23.672	14.855	0.15	0.03	0.46	0.29	8.508	0.157
acc_fac=2.0	5.029	2.962	22.769	14.824	0.12	0.03	0.39	0.31	8.421	0.144
bb_factor=1	5.114	3.015	23.673	14.825	0.14	0.03	0.44	0.33	8.577	0.157
bb_factor=10	5.193	2.967	23.184	14.830	0.14	0.03	0.49	0.32	8.531	0.160

astar_fac=1.0	5.031	2.968	23.360	14.813	0.14	0.03	0.40	0.28	8.478	0.147
astar_fac=2.0	5.140	2.988	23.080	14.912	0.15	0.03	0.42	0.38	8.527	0.164
max_crit=0.5	5.497	3.064	23.424	15.425	0.13	0.02	0.28	0.27	8.832	0.118
max_crit=1.0	5.025	2.975	22.702*	14.961*	0.15	0.03	0.59	0.38	8.441	0.178
crit_exp=0.5	5.065	2.971	23.168	14.835	0.16	0.03	0.42	0.38	8.480	0.166
crit_exp=2.0	5.087	2.967	23.379	14.829	0.12	0.03	0.33	0.26	8.505	0.133
convergence=20	5.032	2.961	22.669	14.830	0.22	0.04	0.53	0.27	8.413	0.188
convergence=100	5.032	2.961	22.669	14.830	0.22	0.04	0.53	0.27	8.413	0.188

Section 6.4 #3

Create a chart or table showing the data. Show the data averaged over seeds for each benchmark. Also, show the data averaged over all benchmarks. Comment on how router run-time and quality is affected by disabling A* routing.



Disabling A* routing (astar_fac=0) converts the path search from A* to Dijkstra's algorithm. Run-time increases significantly (sha: ~10x, diffeq1: ~3.6x) because without the heuristic, the router explores many more nodes. Quality is comparable or slightly better since Dijkstra finds the true shortest path. The tradeoff is clear: A* provides a massive speedup with minimal quality loss, making it essential for practical FPGA routing.

Benchmark	A* enabled delay (ns)	A* enabled runtime (s)	A* disabled delay (ns)	A* disabled runtime (s)
alu4	5.497	0.14	5.340	0.36

ex4p	3.064	0.03	3.049	0.03
diffeq1	23.424	0.31	23.647	1.13
sha	15.425	0.21	15.278	2.04
GEOMEAN	8.832	0.129	8.758	0.397